PrO: Latent Proprioceptive Odometry for Goal-Reaching via Real-World Reinforcement Learning

Zhaoyang Cheng[1*], Hongde Hao[1*], Mengjie Zhao[1],
Shaoting Zhu[1,2†], Hang Zhao[1,2†]
[1]Institute for Interdisciplinary Information Sciences, Tsinghua University
[2]Shanghai Qi Zhi Institute
[*]Equal contribution [†]Equal advising

Abstract: Odometry is fundamental to autonomous navigation, goal reaching, and motion control for legged robots. However, conventional filter-based methods suffer from continuous drift caused by modeling errors, while learning-based methods are affected by accumulated prediction errors and the sim-to-real gap. In this paper, we propose proprioceptive odometry tracking (PrO), a proprioceptive-only odometry tracking method for goal-reaching tasks with legged robots. PrO employs a low-dimensional latent representation supervised by the displacement between adjacent time steps and unifies odometry estimation and action generation, thereby reducing the effects of accumulated prediction errors and the cross-domain gap. We further propose a two-stage training method in which a locomotion policy is first pre-trained in IsaacLab and then post-trained in the real-world. This enables the odometry representation to adapt to real-world feedback relevant to goal-reaching tasks. Extensive experiments show that PrO achieves lower average distance and heading errors in both sim-to-sim and sim-to-real transfers.

Keywords: Legged Robots, Localization, Reinforcement Learning

1 Introduction

Odometry provides robots with continuous estimates of pose and motion and serves as a foundation for autonomous navigation, path planning, and motion control. For legged robots, accurate odometry estimation not only describes the robot's motion relative to its starting point, but also directly affects goal reaching and velocity tracking [1, 2]. In environments without external sensors, obtaining reliable odometry using only proprioception [3, 4] is particularly important because exteroceptive sensors such as camera or LiDAR may be affected by occlusions and environmental structure.

Previous methods still face multiple challenges. SLAM-based methods can estimate motion from image sequences [5, 6] or LiDAR [7, 8], but their tracking performance can readily degrade when a legged robot moves rapidly or turns sharply, or when visual conditions are poor. Conventional proprioceptive odometry generally uses filters to fuse measurements from IMUs, joint encoders, and contact information, but modeling errors can lead to continuous drift [9, 10, 3]. AutoOdom introduced learning-based autoregressive proprioceptive odometry and mitigated sim-to-real gap through simulation pre-training and augmentation with limited real-world data [4]. However, it remains affected by accumulated prediction errors and a large sim-to-real gap.

We avoid directly using the explicit position difference $\Delta\mathbf{p}$ as an intermediate representation. We find that this explicit variable is directly regressed against ground truth, which differs substantially between simulation and reality. Also, its prediction errors are propagated

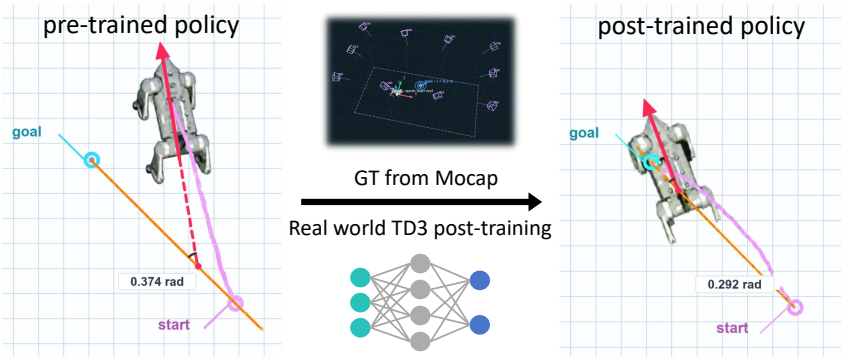


Figure 1: Proprioceptive odometry tracking for legged robots. We first pre-train the policy in IsaacLab with PPO algorithm. By utilizing TD3 to perform post-training in the real world, our framework enables the robot to reach the commanded goal pose more accurately and follow smooth trajectories, turning failure (left) into success (right).

directly to subsequent decisions and may continuously accumulate during long-horizon execution, potentially leading to a large sim-to-real gap. In contrast, a low-dimensional latent representation can preserve odometry-related motion information under the supervision of $\mathcal{L}_{\Delta p}$. This avoids accumulated errors in autoregressive odometry associated with the explicit $\Delta\mathbf{p}$. Moreover, real-world reinforcement learning can optimize this representation using feedback directly related to goal reaching, further reducing the sim-real gap.

To address the challenges encountered by existing methods, we propose PrO, a proprioceptive-only odometry tracking method. PrO introduces a low-dimensional latent representation encoded from proprioception and a high-level goal command. A lightweight decoder predicts $\Delta\mathbf{p}$, to which the position-supervision loss is applied using ground truth, while the policy head maps this latent representation, proprioception, and high-level goal commands to joint actions. For training, we first obtain a robust pre-trained locomotion policy through large-scale parallel reinforcement learning in IsaacLab and then perform post-training with TD3 in the real world. During post-training, we freeze the policy head M_p and jointly fine-tune the latent encoder. This enables the latent representation to adapt to real-world contact, actuation, latency, and sensor noise while preserving the existing action distribution. In summary, our contributions are as follows:

- We propose PrO, a proprioceptive odometry framework that unifies odometry estimation and action generation through a compact latent representation.
- We develop a two-stage training strategy that pre-trains PrO in simulation and then adapts its odometry representation through real-world reinforcement learning.
- We demonstrate that PrO achieves the highest success rates among the evaluated methods in real-world goal-reaching tasks, providing a practical tool for downstream robotic applications.

2 Related Works

2.1 Odometry and State Estimation for Legged Robots

Odometry is essential for the navigation and locomotion of legged robots. Common approaches include vision-based and proprioceptive odometry methods. SLAM-based methods are among the primary visual approaches used in real-world deployment [5, 6, 7, 8]. Proprioceptive odometry is important for robot state estimation without GPS, cameras, or other external sensors. Filtering-based methods use recursive state-estimation frameworks, such as the Extended Kalman Filter (EKF) or Unscented Kalman Filter (UKF), to

integrate information from IMUs and proprioceptive sensors [11, 9, 10]. Hybrid learning-filtering methods can outperform purely analytical approaches by combining probabilistic filtering with the representational power of neural networks [10]. LSTM-based displacement estimators have demonstrated reductions in odometry drift of up to 64.93% compared with baseline filtering methods [3]. Pure learning-based methods directly learn complex motion distributions from proprioception without relying on explicit kinematic or dynamic models [12]. However, previous work requires large amounts of adequately constrained training data, whereas real-world data collection is time-consuming and robot-intensive [13]. Although simulation provides abundant data, prior methods suffer from the sim-to-real gap, resulting in poor real-world deployment performance [14].

2.2 Learning-Based Navigation and Locomotion

Learning-based legged locomotion initially focused on robustness to challenging terrain and dynamic perturbations. Previous studies use reinforcement learning to adapt to complex terrain, unknown dynamics, and noisy perturbations [15, 16, 17]. Flexible control for legged robots has also been achieved through different model architectures and dynamics priors [18, 19]. In addition, learning-based navigation has been studied across different platforms and embodiments [1, 20, 21].

Many navigation and locomotion systems for legged robots decouple high-level planning from low-level tracking. Some learning-based approaches generate partial navigation paths or scenario-specific strategies [22, 23], which are then executed by a low-level tracker for accurate deployment. With the development of vision-language models, high-level planners can extract semantically informative subtasks [24, 25] and waypoints [2] for low-level execution.

3 PrO: Proprioceptive Odometry Tracking

PrO leverages a neural network to estimate odometry and the next action from a history of proprioceptive joint positions and a high-level goal command. This allows the robot to follow a smooth trajectory toward the commanded pose without external sensors while aligning its final heading with the line segment from the start to the end position.

3.1 Problem Formulation

We consider a finite-horizon goal-reaching task for a quadruped robot using only proprioceptive sensing. The robot aims to reach a commanded position and heading within a specified duration and with a desired terminal speed, while maintaining a stable gait.

Observations: At each discrete time step t, the actor receives an observation combining proprioceptive information and a high-level tracking command:

$$\mathbf{o}_t = [\mathbf{o}_t^p, \mathbf{c}_t]. \tag{1}$$

The proprioceptive input $\mathbf{o}_t^p = (\mathbf{v}_t, \boldsymbol{\omega}_t, \mathbf{g}_t, \mathbf{q}_t, \dot{\mathbf{q}}_t, \mathbf{a}_{t-1})$ contains base linear velocity, base angular velocity, projected gravity, joint positions, joint velocities, and the previous action. The base linear velocity input is set to zero for the actor but retained for the critic. The critic additionally includes a height-scan input $\mathbf{h}_t$, which is fixed to zero in our setting. No camera, LiDAR, or external odometry inputs are provided to the actor.

The high-level tracking command is defined as

$$\mathbf{c}_t = [x_t^*, y_t^*, \psi_t^*, T_t^*, v_F^*, \tau_t]^\mathsf{T}, \tag{2}$$

where (x_t^*, y_t^*) is the target position, ψ_t^* is the desired heading, T_t^* is the desired duration, v_F^* is the terminal speed, and τ_t is the elapsed time.

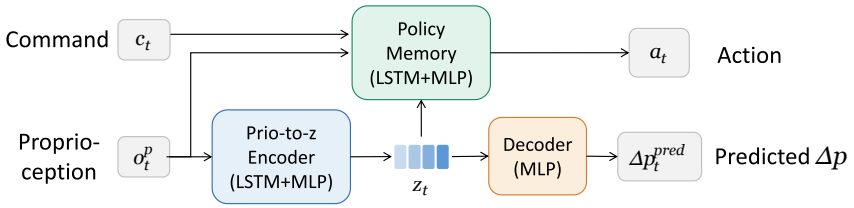


Figure 2: PrO latent-network architecture. Proprioception is encoded into a low-dimensional latent representation $\mathbf{z}_t$. Together with the task command $\mathbf{c}_t$, $\mathbf{z}_t$ is mapped to the joint actions $\mathbf{a}_t$. In addition, the predicted position displacement $\Delta\mathbf{p}_t^{\text{pred}}$, decoded from $\mathbf{z}_t$, is supervised by the ground-truth displacement $\Delta\mathbf{p}_t^{\text{gt}}$ from the simulator using an MSE loss.

Actions: The policy outputs 12 joint-position offsets, $\mathbf{a}_t = \pi_\theta(\mathbf{o}_t) \in \mathbb{R}^{12}$. These offsets are added to the default joint configuration $\mathbf{q}^{\text{default}}$ and tracked by a low-level PD controller, which produces joint torques:

$$\mathbf{M}_t = k_p\big(\mathbf{q}^{\text{default}} + \mathbf{a}_t - \mathbf{q}_t\big) - k_d\dot{\mathbf{q}}_t, \tag{3}$$

where $k_p = 40$ and $k_d = 1$.

3.2 Stage 1: pre-training in Simulation

In the first stage, a Unitree Go2 locomotion policy is pre-trained in IsaacLab [26] using 4096 parallel simulation environments. The rollouts cover diverse commanded poses, headings, durations, and final reaching velocities, while randomization perturbs the initial state, initial speed, contacts, payloads, sensing, and dynamics. This process produces a robust locomotion prior policy and a prior distribution over low-dimensional latent representations.

3.2.1 Policy Architecture

The actor maps proprioception and commands to a motion representation. The proprioception-to-z encoder E_p maps $\mathbf{o}_t^p$ to a 4-dimensional latent representation, $\mathbf{z}_t, \mathbf{h}_t^e = E_p(\mathbf{o}_t^p, \mathbf{h}_{t-1}^e)$. It comprises a single-layer, 256-unit LSTM and a 64-unit projection MLP. The policy head M_p consists of a second 256-unit LSTM and a three-layer MLP with widths of 512, 256, and 128. It receives $[\mathbf{z}_t, \mathbf{o}_t^p, \mathbf{c}_t]$ and predicts joint actions. The critic uses the critic memory M_c, which has the same architecture as M_p, to map privileged observations to values.

A lightweight decoder predicts the planar displacement, $\Delta\mathbf{p}_t^{\text{pred}} = D_\omega(\mathbf{z}_t)$. Its ground-truth target is obtained from simulation, but neither the target nor the decoder is required during execution. This supervision encourages the latent representation to retain motion information relevant to odometry tracking. The architecture is shown in Fig. 2.

3.2.2 Training Method

We optimize the actor–critic using PPO [27]. The actor objective also includes the $\mathcal{L}_{\Delta p}$

$$\mathcal{L}_{\Delta p} = \frac{1}{N}\sum_{t=1}^{N} \|\Delta\mathbf{p}_t^{\text{pred}} - \Delta\mathbf{p}_t^{\text{gt}}\|_2^2, \tag{4}$$

weighted by $\lambda_{\Delta p}$. The overall training objective is

$$\mathcal{L} = \mathcal{L}_{\text{PPO}} + \lambda_{\Delta p}\mathcal{L}_{\Delta p}. \tag{5}$$

The $\mathcal{L}_{\Delta p}$ helps the latent distribution capture information relevant to the odometry-estimation task.

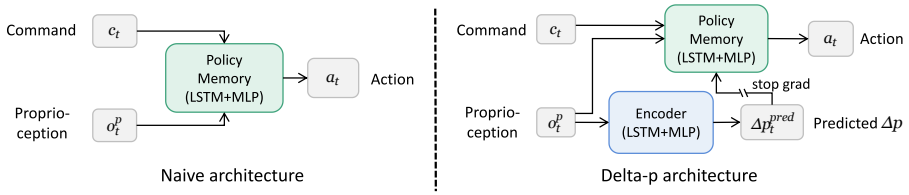


Figure 3: In the Naive architecture, the observation is mapped directly to actions. In the Delta-p architecture, proprioception is encoded into the explicit variable $\Delta \mathbf{p}_t^{\text{pred}}$.

Terminations: A training episode terminates when any of the following conditions is met: (1) Reach Goal: within $\lambda_\tau T_t^*$, the base pose has a distance error of at most 0.3 m from the commanded pose and a heading error of at most 0.3 rad from the line segment connecting the start and end positions, where T_t^* denotes the duration command and λ_τ is the reach-goal time coefficient. The value of λ_τ is 0.8 in simulation and 0.9 in the real world. (2) Task Timeout: the robot does not reach the goal and the elapsed time exceeds $1.01T_t^*$. (3) Bad Orientation: the base orientation exceeds the angle limit.

Rewards: The reward function comprises task r_{task}, regularization r_{reg}, safety r_{safe}, and AMP r_{AMP} terms. We formulate the task as reaching the goal within the required time and at the required speed. To ensure that the robot follows a smooth trajectory, we use a dense velocity-tracking reward to track real-time speed and a dense heading-tracking reward to track the instantaneous heading, rather than relying on a sparse goal reward. The regularization terms penalize the action rate and deviation from a flat orientation to prevent oscillations, as well as velocity-limit and angular-velocity-limit violations to prevent sharp movements and turns. Meanwhile, the safety terms enforce joint and torque limits. Furthermore, we leverage Adversarial Motion Priors (AMP) [28], trained on MPC-generated datasets [29, 30], to encourage natural and robust motion styles. Further details are provided in the Appendix B.

3.3 Stage 2: Adaptation through Post-training in Deployment

When directly deployed in MuJoCo [31] and real-world environments, the policy pre-trained in IsaacLab is affected by differences in contact, actuation, latency, and sensor noise. PrO therefore performs post-training in the target domain using the twin delayed deep deterministic policy gradient (TD3) algorithm [32]. Unlike PPO, TD3 stores rollout transitions in a replay buffer, improving data efficiency in a nonparallel environment.

The actor network is initialized from the pre-trained checkpoint, while the critics are learned from scratch in the real world. We freeze the M_p to prevent drift in the action distribution. The E_p is fine-tuned with an Adam optimizer using a weighted sum of the RL loss and $\mathcal{L}_{\Delta p}$ to learn a more precise latent distribution.

4 Experiments

We evaluate our method on single-point reaching tasks in both simulation and real-world environments, as well as on complex real-world locomotion tasks, to answer the following questions:

(i) Does our proprioception-based method outperform other learning-based and SLAM-based baselines?

(ii) How do our policy architecture and learning strategy contribute to task performance?

(iii) Does post-training with real-world reinforcement learning effectively reduce the sim-to-real gap?

(iv) How does our method support downstream locomotion tasks on real-world quadruped robots?

4.1 Experimental Setup

4.1.1 Robot and Simulation Platforms

The experiments are conducted in three environments. In Stage 1, we train in IsaacLab [26] with 4,096 parallel Unitree Go2 environments on an NVIDIA RTX 3090. The pre-trained policy is then evaluated in an IsaacLab environment containing a single Unitree Go2. In Stage 2, post-training is performed on an NVIDIA RTX 4090D. Sim-to-sim experiments use an independent MuJoCo [31] environment. In the real-world experiments, joint commands are sent to the Go2 through Unitree SDK2, while a motion-capture system establishes the local frame and provides ground truth. Before computing the evaluation metrics, all trajectories are transformed into the coordinate frame defined at the start of each episode.

4.1.2 Evaluation Tasks and Metrics

We use a fixed benchmark of 100 single-goal rollouts per environment, with the same set of commanded poses used across all methods and experiments to ensure consistent comparisons. For each rollout, the robot starts from position (x_0, y_0) with heading $\psi_0 = 0$. The commanded positions are sampled from $(x_t^*, y_t^*) \in [0.0, 2.0] \times [-1.0, 1.0]$ meters, and the desired heading is determined by $\psi_t^* = atan2(y_t^* - y_0, x_t^* - x_0)$. The primary evaluation metric is the success rate (SR), with final position and heading error tolerances of 0.3 m and 0.3 rad, respectively. For each command, both distance error and heading error are derived from the final pose and target pose at the end of the episode. We additionally report the average distance error (ADE) and average heading error (AHE) for 100 rollouts.

4.2 Comparison and Ablation Baselines

4.2.1 Comparison Experiments

- AutoOdom [4]: A representative open-source, learning-based odometry method for quadruped robots that uses no external sensors.
- DROID-SLAM [6]: A representative dense visual SLAM method that provides odometry.

For these two baselines, which predict real-time odometry, we derive the velocity command from the predicted pose and the target pose. Specific details are shown in the Appendix A.

4.2.2 Ablation Study

- Naive: The M_p module directly maps the observation to a 12-dimensional action. The architecture is shown on the left of Fig. 3.
- Delta-p: The encoder maps proprioception to an explicit variable $\Delta\mathbf{p}_t^{\text{pred}}$, which is passed to the M_p module together with the observation. The architecture is shown on the right of Fig. 3.
- w/o $\mathcal{L}_{\Delta p}$: We omit the $\mathcal{L}_{\Delta p}$ from our training method. The E_p module is optimized only by the RL loss.
- w/o post-training: We directly roll out the pre-trained policy in the target domain (MuJoCo and the real world) without any post-training.
- w/o rl loss: During post-training, E_p is optimized only with $\mathcal{L}_{\Delta p}$, without the TD3 actor loss.

- Full-finetune: During post-training, both E_p and the policy head M_p are optimized jointly, instead of freezing M_p as in PrO.

4.3 Results of the Pre-trained Policy

This experiment compares PrO with two ablation architectures, Naive and Delta-p. As shown in Table 1, PrO achieves the highest success rate in the Stage 1 simulation. This indicates that our low-dimensional latent design avoids the error accumulation associated with the explicit $\mathbf{\Delta p}$ in the Delta-p architecture.

Table 1: Architecture Ablations in IsaacLab

Architecture	Naive	Delta-p	PrO (Ours)
SR ↑	80%	94%	96%

4.4 Results of the Post-trained Policy

The following experiments compare PrO with the directly deployed pre-trained policy and with policies post-trained in the target domain using ablated architectures and training methods. We also compare PrO with AutoOdom [4] and DROID-SLAM [6] in the real-world environment.

4.4.1 Sim-to-Sim Transfer to MuJoCo

Table 2: Success Rates with Architecture and Training Ablations for Isaac-to-MuJoCo

Architecture	SR ↑			
	w/o post-training	post-trained	w/o $\mathcal{L}_{\Delta p}$	full-finetune
Delta-p	55%	63%	-	27%
PrO (Ours)	83%	91%	90%	25%

Table 2 shows that PrO ultimately achieves a success rate 28% higher than that of the ablated Delta-p architecture. This is because the explicit $\mathbf{\Delta p}_t^{\text{pred}}$ is more severely affected by the sim-to-sim gap than our latent design. Furthermore, post-training improves the success rate by 8%, reducing the sim-to-sim gap. The freezing strategy is more effective than full fine-tuning, where full fine-tuning achieves a success rate of only 25%. This is because full fine-tuning causes the model to forget the motion prior. These results demonstrate that both our architecture and training method contribute to improved sim-to-sim performance.

In addition, Table 3 shows that the PrO architecture achieves lower ADE and AHE than the Delta-p architecture in sim-to-sim transfer. This directly demonstrates that our latent design mitigates error accumulation in both distance and heading. Our post-trained policy reduces both ADE and AHE compared to the policy without post-training, but w/o $\mathcal{L}_{\Delta p}$ fails to do so. This is because omitting $\mathcal{L}_{\Delta p}$ leaves the latent representation less constrained and leads to larger errors.

4.4.2 Sim-to-Real Transfer

The sim-to-real success rates of PrO are shown in Table 4 and Table 5. Post-training yields a 10% improvement in sim-to-real transfer, reducing the sim-to-real gap in position. PrO outperforms AutoOdom by 57%. AutoOdom turns out to be poor because it suffers from error accumulation caused by explicit Δp, and supervised learning is limited by insufficient real-world data. Moreover, PrO outperforms DROID-SLAM by 26%. SLAM-based method is less satisfactory due to the instability of the robot-SLAM system.

Table 3: Architecture and Training Ablations for Isaac-to-MuJoCo

Architecture	Training Method	SR ↑	ADE (m)↓	AHE (rad)↓
Delta-p	w/o post-training	55%	0.454	0.160
Delta-p	full-finetune	27%	0.368	0.287
Delta-p	post-trained	63%	0.430	0.172
PrO	w/o post-training	83%	0.241	0.161
PrO	w/o $\mathcal{L}_{\Delta p}$	90%	0.224	0.189
PrO	w/o rl-loss	56%	0.311	0.195
PrO	full-finetune	25%	0.446	0.159
PrO	post-trained (Ours)	91%	0.235	0.159

Table 4: Comparison with Baselines for Isaac-to-Real

	AutoOdom [4]	DROID-SLAM [6]	PrO (Ours)
SR ↑	14%	45%	71%

In real-world deployment, our method achieves lower ADE and AHE than the policy without post-training, as shown in Table 5. This directly demonstrates that our post-training method reduces the sim-to-real gap through reinforcement learning. Although w/o $\mathcal{L}\Delta p$ achieves the lowest AHE, it does so at the cost of a higher ADE, as omitting $\mathcal{L}\Delta p$ removes the odometry constraint on the latent distribution.

4.5 Results of the Downstream task

In this experiment, we use PrO as a low-level tracker for VLM inference. Roadblocks of different colors are placed in the environment, and our instruction is to reach the red roadblock. We use NaVILA [25] to identify and locate the red roadblock. The location will be used to generate a pose command with depth estimation. We then use PrO to follow the command and reach the red roadblock. The pair of NaVILA and PrO can achieve our goal in various environment setups with precision, as shown in Fig. 4.

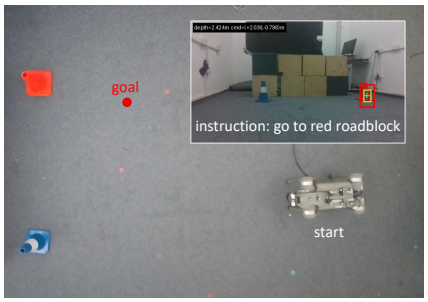


Figure 4: The overhead view of the command generated in a downstream task. The top right image is the inference result of NaVILA.

5 Conclusion

In this paper, we present PrO, a proprioceptive odometry tracking method that unifies odometry estimation and action generation through a compact, displacement-supervised latent representation. PrO combines simulation pre-training with real-world reinforcement learning to reduce the sim-to-real gap. Experiments demonstrate improved real-world goal-reaching success rates and reduced distance and heading errors.

Table 5: Training Ablations for Isaac-to-Real

Training Method	SR ↑	ADE (m)↓	AHE (rad)↓
w/o post-training	61%	0.278	0.217
w/o $\mathcal{L}_{\Delta p}$	<u>68%</u>	0.311	0.192
w/o rl-loss	58%	0.333	0.243
full-finetune	65%	<u>0.277</u>	0.229
post-trained (Ours)	71%	0.256	<u>0.214</u>

6 Limitations

Despite these advances, there are still several limitations. First, our evaluation covers only a narrow range of goal commands, leaving generalization to more distant goals and longer task horizons insufficiently validated. Moreover, PrO is primarily evaluated on obstacle-free, flat terrain, with limited exploration of more complex environments. Future work will address these limitations through broader goal sampling, curriculum learning, and richer environment randomization, including obstacle-aware goal-reaching tasks.

Appendix

A Velocity Command Derivation from Comparison Baselines

For AutoOdom[4] and DROID-SLAM[6] baselines, the estimated robot pose is converted into velocity commands using a PD controller. Specifically, given the predicted position $\hat{\mathbf{p}}_t$ and the target position $\mathbf{p}^{\text{goal}}$, the position error is computed as

$$\mathbf{e}_t^p = \mathbf{p}^{\text{goal}} - \hat{\mathbf{p}}_t. \tag{6}$$

The commanded linear velocity is then given by

$$\mathbf{v}_t^{\text{cmd}} = k_p \mathbf{e}_t^p - k_d \dot{\mathbf{v}}_t, \tag{7}$$

where $\hat{\mathbf{v}}_t$ denotes the estimated robot velocity, obtained from consecutive odometry estimates as

$$\hat{\mathbf{v}}_t = \frac{\hat{\mathbf{p}}_t - \hat{\mathbf{p}}_{t-1}}{\Delta t}. \tag{8}$$

Similarly, the commanded angular velocity is computed from the heading error:

$$e_t^\psi = \text{wrap}\left(\psi^{\text{goal}} - \hat{\psi}_t\right), \tag{9}$$

$$\omega_t^{\text{cmd}} = k_p^\psi e_t^\psi - k_d^\psi \hat{\omega}_t. \tag{10}$$

The resulting linear and angular velocity commands are directly sent to the robot's low-level locomotion controller.

B Reward Terms

Let $\mathbf{p}_t = (x_t, y_t)^\mathsf{T}$, $\mathbf{p}_t^* = (x_t^*, y_t^*)^\mathsf{T}$, $d_t = \|\mathbf{p}_t - \mathbf{p}_t^*\|_2$, and $\Delta\mathbf{p}^* = \mathbf{p}_t^* - \mathbf{p}_0$. The desired average and final velocities are $\bar{\mathbf{v}}^* = \Delta\mathbf{p}^*/(T_t^* + \epsilon)$ and $\mathbf{v}_F^* = v_F^* \Delta\mathbf{p}^*/(\|\Delta\mathbf{p}^*\|_2 + \epsilon)$, respectively. We define $\ell_t = \|(\mathbf{p}_t - \mathbf{p}_0) - \Delta\mathbf{p}^*\|_2/(\|\Delta\mathbf{p}^*\|_2 + \epsilon)$ and $\psi_t^{\text{ref}} = \psi_0 + \text{wrap}(\phi^*)\tau_t/(T_t^* + \epsilon)$, where $\phi^* = \text{atan2}(\Delta p_y^*, \Delta p_x^*)$. Here, $\mathbb{I}(\cdot)$ denotes the indicator function, $(x)_+ = \max(x, 0)$, $\text{wrap}(\cdot)$ maps an angle to $[-\pi, \pi]$, and $\sigma = 0.5$. The variable $c_{f,t}$ indicates the first contact of foot f, and $D(\mathbf{s}_t, \mathbf{s}_{t+1})$ is the AMP discriminator score. For the goal-distance progress term, d_{t-1} is initialized to d_t at the first step. The complete reward specification is given in Table 6.

C Training Details

Table 7 includes details for simulation and PPO training in Stage 1, as well as TD3 training in Stage2.

Table 6: Reward Terms and Details

Reward Term	Weight	Mathematical Formulation
Task Reward		
Goal Distance Progress (r_{prog})	200	$\mathbb{I}(\tau_t < T_t^*)\,\mathbb{I}(d_t > 0)\,(d_{t-1} - d_t)$
Goal Heading ($r_{\text{goal-head}}$)	4	$-\mathbb{I}(\tau_t \geq 0.9T_t^*)\,\lvert\,\text{wrap}(\psi_t - \psi_t^*)\rvert$
Final Velocity Tracking ($r_{\text{final-vel}}$)	10	$\mathbb{I}(0.8T_t^* \leq \tau_t < T_t^*)\exp\Big(-\frac{\lVert\mathbf{v}_{t,xy} - \mathbf{v}_F^*\rVert_2^2}{\sigma^2}\Big)$
Linear Velocity Tracking (r_{vel})	10	$\mathbb{I}(\ell_t > 0.1)\exp\Big(-\frac{\lVert\mathbf{v}_{t,xy} - \bar{\mathbf{v}}^*\rVert_2^2}{\sigma^2}\Big)$
Heading Tracking (r_{head})	14	$\exp\Big(-\frac{\text{wrap}(\psi_t^{\text{ref}} - \psi_t)^2}{\sigma^2}\Big)$
Timed Goal Reach (r_{reach})	10	$\mathbb{I}(0.8T_t^* \leq \tau_t < T_t^*)\,\mathbb{I}(d_t < 0.4)$
Task Timeout (r_{timeout})	1	$-\mathbb{I}(\tau_t \geq T_t^*)$
Is Alive (r_{alive})	5	$\mathbb{I}$(no undesired termination)
Is Terminated (r_{term})	1	$-\mathbb{I}$(undesired termination)
Regularization Reward		
Vertical Velocity (r_{v_z})	2	$-v_{t,z}^2$
Roll–Pitch Angular Velocity ($r_{\omega_{xy}}$)	0.05	$-\lVert\boldsymbol{\omega}_{t,xy}\rVert_2^2$
Action Rate ($r_{\Delta a}$)	0.01	$-\lVert\mathbf{a}_t - \mathbf{a}_{t-1}\rVert_2^2$
Linear Velocity Limit ($r_{v\text{-lim}}$)	1	$-\mathbb{I}(\lVert\mathbf{v}_{t,xy}\rVert_2 > 1.5)$
Yaw Velocity Limit ($r_{\omega\text{-lim}}$)	1	$-\mathbb{I}(\lvert\omega_{t,z}\rvert > 1.0)$
Joint Acceleration ($r_{\ddot{q}}$)	2.5×10^{-7}	$-\lVert\ddot{\mathbf{q}}_t\rVert_2^2$
Joint Velocity ($r_{\dot{q}}$)	3×10^{-4}	$-\lVert\dot{\mathbf{q}}_t\rVert_2^2$
Feet Air Time (r_{air})	0.01	$\sum_f c_{f,t}\left[t_f^{\text{air}} - 0.3 + 10\min(0.5 - t_f^{\text{air}}, 0)\right]$
Flat Orientation (r_{ori})	0.05	$-\lVert\mathbf{g}_{t,xy}\rVert_2^2$
Joint Torque (r_M)	2×10^{-4}	$-\lVert\mathbf{M}_t\rVert_2^2$
Joint Deviation (r_q)	0.01	$-\lVert\mathbf{q}_t - \mathbf{q}^{\text{def}}\rVert_2^2$
Hip Joint Deviation ($r_{q,\text{hip}}$)	1	$-\sum_{j\in\mathcal{J}_{\text{hip}}}(q_{j,t} - q_j^{\text{def}})^2$
Safety Reward		
Joint Position Limit ($r_{q\text{-lim}}$)	0.75	$-\sum_j\left[\mathbb{I}(q_{j,t} < q_j^-) + \mathbb{I}(q_{j,t} > q_j^+)\right]$
Joint Velocity Limit ($r_{\dot{q}\text{-lim}}$)	0.02	$-\sum_j \text{clip}(\lvert\dot{q}_{j,t}\rvert - 0.8\dot{q}_j^{\max}, 0, 1)$
Joint Torque Limit ($r_{M\text{-lim}}$)	0.1	$-\sum_j\left(\lvert M_{j,t}\rvert - 0.8M_j^{\max}\right)_+^2$
AMP Reward		
Adversarial Motion Prior (r_{AMP})	0.5	$\max\left\{0, 1 - 0.25\left[D(\mathbf{s}_t, \mathbf{s}_{t+1}) - 1\right]^2\right\}$

Table 7: Training Details for Stage 1 and Stage 2

Stage	Training Detail	Setting
Stage 1	Simulation frequency	200 Hz
	Action decimation / control frequency	4 / 50 Hz
	Joint-target scale around the default pose	0.5
	Transitions per environment per PPO iteration	50
	PPO epochs / mini-batches per epoch	5 / 4
	Maximum training iterations	10,000
	Adaptive learning rate	10^{-3}
	Discount coefficient	0.99
	GAE parameter	0.95
	Target KL divergence	0.01
	$\mathcal{L}_{\Delta p}$ coefficient $\lambda_{\Delta p}$	0.1
Stage 2	TD3 replay-buffer capacity	10^5
	Sequence length	24
	Sequence batch size	32
	Exploration noise	0.15
	Actor / critic learning rate	10^{-7} / 10^{-5}
	$\mathcal{L}_{\Delta p}$ coefficient $\lambda_{\Delta p}$	Same as in Stage 1

Acknowledgments

The authors of this paper used ChatGPT to remove the background floor of the images on both sides of Fig. 1, and replaced it with the grid.

References

[1] J. Truong, D. Yarats, T. Li, F. Meier, S. Chernova, D. Batra, and A. Rai. Learning navigation skills for legged robots with learned robot embeddings. In 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 484–491. IEEE, 2021.

[2] Z. Xu, A. H. Raj, X. Xiao, and P. Stone. Dexterous legged locomotion in confined 3d spaces with reinforcement learning. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 11474–11480. IEEE, 2024.

[3] R. Buchanan, M. Camurri, F. Dellaert, and M. Fallon. Learning inertial odometry for dynamic legged robot state estimation. In Conference on robot learning, pages 1575–1584. PMLR, 2022.

[4] C. Luo, Y. Wang, W. Cai, and M. Zhao. Autoodom: Learning auto-regressive proprioceptive odometry for legged locomotion. In 2025 IEEE International Conference on Robotics and Biomimetics (ROBIO), pages 2301–2306. IEEE, 2025.

[5] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos. Orb-slam: A versatile and accurate monocular slam system. IEEE transactions on robotics, 31(5):1147–1163, 2015.

[6] Z. Teed and J. Deng. Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras. Advances in neural information processing systems, 34:16558–16569, 2021.

[7] W. Xu and F. Zhang. Fast-lio: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter. IEEE Robotics and Automation Letters, 6(2): 3317–3324, 2021.

[8] D. He, W. Xu, N. Chen, F. Kong, C. Yuan, and F. Zhang. Point-lio: robust high-bandwidth light detection and ranging inertial odometry. Advanced Intelligent Systems, 5(7):2200459, 2023.

[9] M. Bloesch, C. Gehring, P. Fankhauser, M. Hutter, M. A. Hoepflinger, and R. Siegwart. State estimation for legged robots on unstable and slippery terrain. In 2013 IEEE/RSJ International Conference on Intelligent Robots and Systems, pages 6058–6064. IEEE, 2013.

[10] M. Camurri, M. Ramezani, S. Nobili, and M. Fallon. Pronto: A multi-sensor state estimator for legged robots in real-world scenarios. Frontiers in Robotics and AI, 7:68, 2020.

[11] G. Welch, G. Bishop, et al. An introduction to the kalman filter. 1995.

[12] C. Chen, X. Lu, A. Markham, and N. Trigoni. Ionet: Learning to cure the curse of drift in inertial odometry. In Proceedings of the AAAI conference on artificial intelligence, volume 32, 2018.

[13] Y. Gu, Z. Liu, and T. Zou. Enhancing leg odometry in legged robots with learned contact bias: An lstm recurrent neural network approach. In 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 6832–6839. IEEE, 2024.

[14] M. Camurri, M. Fallon, S. Bazeille, A. Radulescu, V. Barasuol, D. G. Caldwell, and C. Semini. Probabilistic contact estimation and impact detection for state estimation of quadruped robots. IEEE Robotics and Automation Letters, 2(2):1023–1030, 2017.

[15] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter. Learning quadrupedal locomotion over challenging terrain. Science robotics, 5(47):eabc5986, 2020.

[16] A. Kumar, Z. Fu, D. Pathak, and J. Malik. Rma: Rapid motor adaptation for legged robots. arXiv preprint arXiv:2107.04034, 2021.

[17] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter. Learning robust perceptive locomotion for quadrupedal robots in the wild. Science robotics, 7 (62):eabk2822, 2022.

[18] R. Huang, S. Zhu, Y. Du, and H. Zhao. Moe-loco: Mixture of experts for multitask locomotion. In 2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 14218–14225. IEEE, 2025.

[19] R. Huang, H. Balim, H. Yang, and Y. Du. Flexible locomotion learning with diffusion model predictive control. arXiv preprint arXiv:2510.04234, 2025.

[20] D. Shah, A. Sridhar, N. Dashora, K. Stachowicz, K. Black, N. Hirose, and S. Levine. Vint: A foundation model for visual navigation. arXiv preprint arXiv:2306.14846, 2023.

[21] A. Sridhar, D. Shah, C. Glossop, and S. Levine. Nomad: Goal masked diffusion policies for navigation and exploration. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 63–70. IEEE, 2024.

[22] P. Roth, J. Nubert, F. Yang, M. Mittal, and M. Hutter. Viplanner: Visual semantic imperative learning for local navigation. In 2024 IEEE International Conference on Robotics and Automation (ICRA), pages 5243–5249. IEEE, 2024.

[23] Z. Gong, Y. Zhong, Y. Ding, T. Hu, G. Zhao, L. Kong, R. Li, J. You, and J. Liang. Flux: Accelerating cross-embodiment generative navigation policies via rectified flow and static-to-dynamic learning. arXiv preprint arXiv:2603.12806, 2026.

[24] S. Zhu, D. Li, L. Mou, Y. Liu, N. Xu, and H. Zhao. Saro: Space-aware robot system for terrain crossing via vision-language model. In 2025 IEEE International Conference on Robotics and Automation (ICRA), pages 14820–14827. IEEE, 2025.

[25] A.-C. Cheng, Y. Ji, Z. Yang, Z. Gongye, X. Zou, J. Kautz, E. Bıyık, H. Yin, S. Liu, and X. Wang. Navila: Legged robot vision-language-action model for navigation. arXiv preprint arXiv:2412.04453, 2024.

[26] M. Mittal, P. Roth, J. Tigue, A. Richard, O. Zhang, P. Du, A. Serrano-Munoz, X. Yao, R. Zurbrügg, N. Rudin, et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. arXiv preprint arXiv:2511.04831, 2025.

[27] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.

[28] X. B. Peng, Z. Ma, P. Abbeel, S. Levine, and A. Kanazawa. Amp: Adversarial motion priors for stylized physics-based character control. ACM Transactions on Graphics (ToG), 40(4):1–20, 2021.

[29] G. Bledt, M. J. Powell, B. Katz, J. Di Carlo, P. M. Wensing, and S. Kim. Mit cheetah 3: Design and control of a robust, dynamic quadruped robot. In 2018 IEEE/RSJ international conference on intelligent robots and systems (IROS), pages 2245–2252. IEEE, 2018.

[30] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim. Dynamic locomotion in the mit cheetah 3 through convex model-predictive control. In 2018 IEEE/RSJ international conference on intelligent robots and systems (IROS), pages 1–9. IEEE, 2018.

[31] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In 2012 IEEE/RSJ international conference on intelligent robots and systems, pages 5026–5033. IEEE, 2012.

[32] S. Fujimoto, H. Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In International conference on machine learning, pages 1587–1596. Pmlr, 2018.